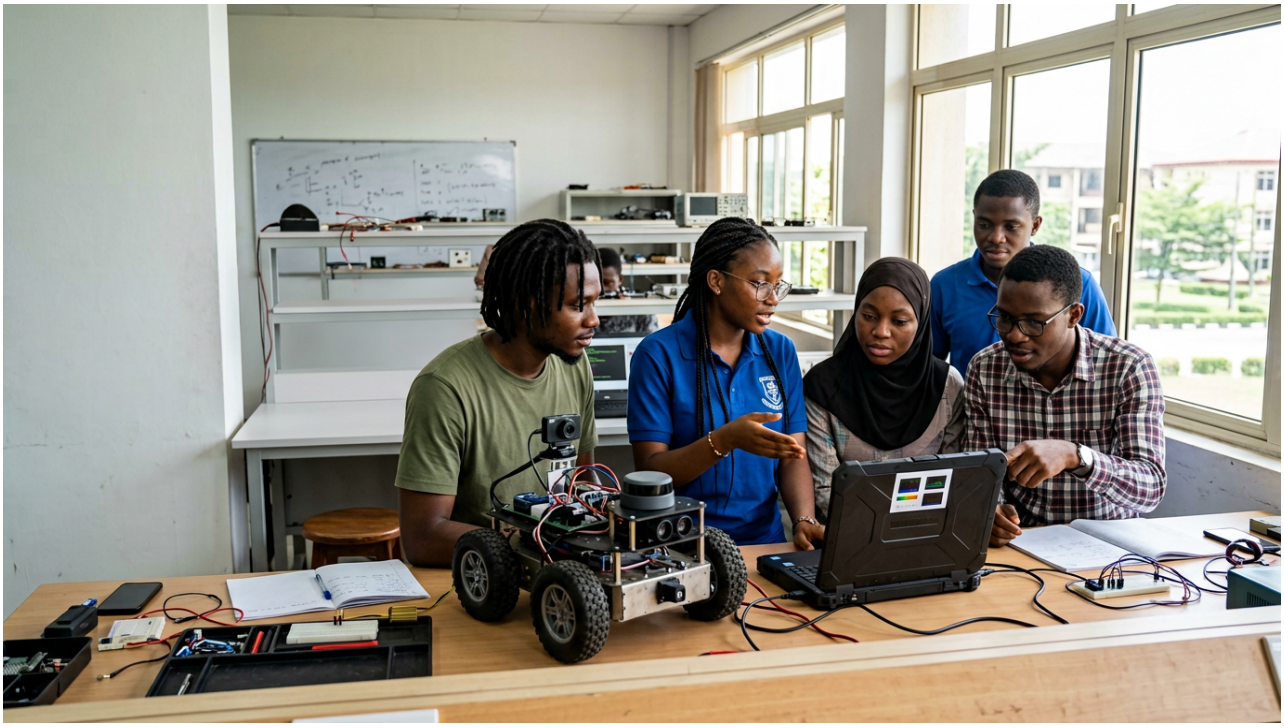
AI.101 — Introduction to Artificial Intelligence



Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.

Narration manuscript · Module 4: Knowledge Representation and Reasoning

Academic level: Bachelor of Artificial Intelligence and Robotics Engineering
Content policy: No faith-related content.

1. Why represent knowledge?

Search gives an agent a way to explore possible action sequences. But search alone does not tell us what a state means, what facts are relevant, or which conclusions follow from observations. For those tasks, an AI system needs a representation: a structured way of encoding information so that a program can store it, update it, and reason with it.

A representation is not a transparent copy of the world. It selects some distinctions and ignores others. A map represents rooms and paths, but may omit surface condition, crowding or access permissions. A database row may record a door as “closed,” but omit whether the sensor is current. A rule system may represent “if the battery is low, return to charging,” but fail to say what happens when the battery sensor is unavailable. Representation choices determine which questions the system can answer and which errors it can make.

We will distinguish four terms. **Data** are recorded values or symbols. A sensor measurement of 18.4 degrees Celsius is data. A **fact** is a statement treated as true within a model, such as “the measured temperature is below the target range.” A **rule** expresses a conditional relationship, such as “if the measured temperature is below the lower threshold, request heating.” A **model** is the larger structured description that connects entities, facts, assumptions and rules.

In practice, a system may also need to represent the source and reliability of a statement. “Door D is closed” could come from a sensor reading made one second ago, a database updated yesterday, or a human report. Treating these as identical facts discards important context. Mature systems often track timestamps, confidence, provenance, and uncertainty. We will develop uncertainty models in a later module; for now, note that a representation must match the questions the application needs to answer.

2. Propositions and logical statements

A simple representation uses propositions: statements that are either true or false within a particular model. For example, let `DoorClosed` mean that a door is closed, and let `RouteBlocked` mean that the robot’s route is blocked. A rule could state: if `DoorClosed` and the route passes through that door, then `RouteBlocked`.

In propositional logic, we can express “if A and B, then C” as `(A AND B) → C`. The arrow means that whenever A and B are true, the rule licenses C. It does not mean that C is the only possible consequence, and it does not describe how the door physically closed. It describes a logical relationship encoded in the system.

This distinction between a statement and its cause matters. A rule system can derive “route blocked” from the known facts, but it does not automatically know whether the door is blocked because a person closed it, a fault occurred, or a sensor sent an incorrect value. A rule can only use what the knowledge base contains.

Propositional representations are clear for small problems, but they can become awkward when we have many objects. If a building contains fifty doors, we do not want to write a separate unrelated proposition for every property of every door. We need a way to represent objects and relations systematically.

3. Objects, predicates and relational structure

A relational representation describes objects and relationships. We might write `Door(D7)`, `Closed(D7)`, `Adjacent(CorridorA, D7)`, and `RequiresCredential(D7)`. A rule can then refer to any door: if a corridor is adjacent to a closed door and the door is required for passage, then that corridor transition is currently unavailable.

Knowledge graph with nodes Robot-7, Corridor-A, Door-7 and Reception. Label relationships “located-at,” “connected-to,” “closed,” and “goal.” A side legend distinguishes sensed facts from inferred relations.

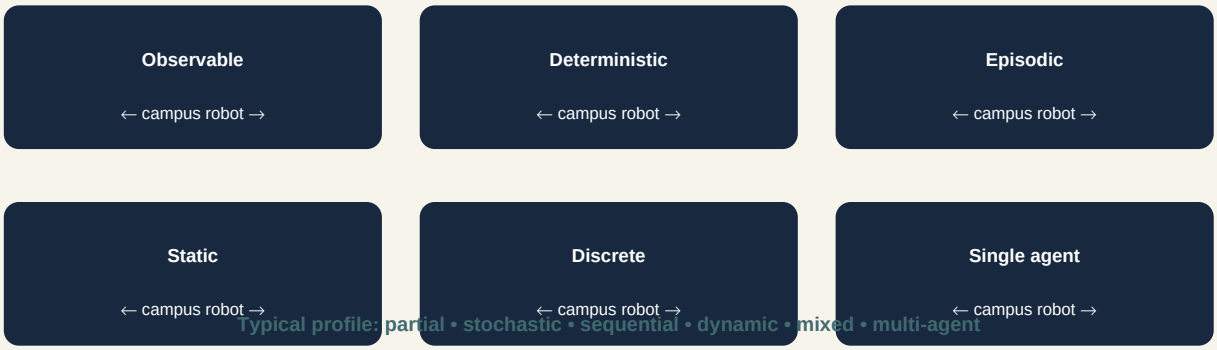


Diagram: Environment classification profile

A knowledge graph uses nodes for entities and edges for relations. Its visual form is useful because it makes connections visible. But the graph is only as reliable as its definitions and data. If the edge `connected-to` is treated as symmetric when it should be one-way, a planner may assume a route exists in the wrong direction. If the fact that a door is closed is stale, the graph may describe a past state rather than the current environment.

A relational system also needs identifiers. “The door near the laboratory” may be ambiguous. A stable identifier such as `D7` lets other facts refer to the same object. Names, types and relations must be consistent across sensors and software components. In robotics, mismatched coordinate frames or inconsistent object identifiers can cause a system to combine facts about different things.

4. Rules and inference

An inference rule describes how the system can derive a new statement from statements it already has. Consider a simple safety assistant for a mobile robot:

- Fact: the battery estimate is below the reserve threshold.
- Fact: the robot is not at a charging station.
- Rule: if the battery is below reserve and the robot is not at a charging station, then the robot should pause new delivery tasks and plan to reach a charging station.

The rule is not the whole safety system. It depends on the battery estimate, the threshold definition, the availability of charging stations, and a valid route planner. Still, the example shows how explicit rules can make some engineering decisions inspectable. An operator can examine the conditions that caused the recommendation.

Forward chaining begins with known facts and repeatedly applies rules that match them. If a new fact is derived, it may activate another rule. For example, a sensor reports low battery; the system derives “reserve condition”; that condition activates the rule to defer new work; a further rule may select a charging route.

Backward chaining begins with a question or goal and works backward to identify what evidence would support it. If the system asks “Can the robot safely take corridor C?”, it may check whether the corridor is connected, whether the door is open, whether the width is sufficient, and whether the route satisfies current safety constraints. If a required fact is missing, the system can report that it cannot determine the answer rather than inventing one.

Two parallel animations. Forward chaining: facts at the left trigger successive rules toward a recommendation. Backward chaining: a question at the right expands into premises that must be checked.

Forward chaining is useful when new facts arrive continuously and we want to update consequences. Backward chaining is useful when we have a specific question and want to retrieve only relevant evidence. Some expert systems combine both strategies.

5. Inconsistency, missing facts and open-world questions

A real knowledge base can contain conflicting information. A building map may say a door is open, while a live sensor reports it closed. Two sensors may disagree. A robot should not silently resolve every conflict by trusting whichever record was loaded last. The system needs a policy: prioritize fresh measurements, request confirmation, represent both claims with their sources, or choose a safe action while uncertainty remains.

Missing information is different from a false statement. If the knowledge base does not contain “door D7 is open,” that does not necessarily mean the door is closed. It may mean no one has checked. Some simple logic systems use a **closed-world assumption**: if a fact is not in the database, treat it as false. That can be suitable for a complete inventory table but unsafe for an incomplete map. An **open-world assumption** allows facts to remain unknown until evidence is available.

For a robot moving through a building, treating “unknown” as “clear” could cause a collision or an attempted passage through a closed door. Treating every unknown fact as “blocked” could make the robot stop forever. The design must distinguish true, false and unknown where the application requires it, and provide a procedure for resolving unknowns.

Rules can also conflict. One rule may say “continue to destination if the route is available,” while another says “stop if a safety sensor is uncertain.” Priority, precedence and exception handling must be defined. The order in which software happens to evaluate rules should not become an accidental safety policy.

6. Worked example: diagnosing a blocked route

Suppose Robot-7 is travelling through Corridor-A toward Reception. The knowledge base contains these facts: Robot-7 is located in Corridor-A; Corridor-A connects to Door-7; Door-7 is reported closed by a sensor updated at the current time; and the planned route requires Door-7. A rule states that if a required door is closed, the route segment is blocked.

Forward chaining applies the rule and derives that the current route segment is blocked. Another rule says that when a planned segment is blocked, the planner should look for an alternate route. The planner then queries the map for a path that avoids Door-7. If it finds one, the system can compare cost and safety. If it does not, it may wait, request assistance or return to a safe location.

Now consider a variation. The door reading is ten minutes old. The system should not treat it as equally current. Add a timestamp and a freshness condition: the rule only accepts a door state if the observation is recent enough for this task. If it is stale, the agent can request a new measurement, approach a safe viewing position, or treat the route as uncertain.

Consider another variation. The map says Door-7 is not part of the corridor, while the live camera indicates a doorway across the path. We may have a map error or a perception error. A rule system cannot determine which source is correct unless we give it evidence and a resolution policy. The appropriate response might be to stop and ask for verification, not to force an artificial certainty.

This example shows both the strength and limitation of symbolic reasoning. Rules make decision paths explicit and testable. But the knowledge base must represent relevant objects, sources, time and exceptions. Symbolic reasoning is powerful when the problem structure is known; it is brittle when its knowledge is incomplete or its assumptions do not match reality.

7. Practical lab: a small rule-based expert system

Build a rule-based assistant that recommends one of three actions for a simulated delivery robot: continue, replan, or request human assistance. Use facts for route clear, battery reserve, sensor confidence, and network availability. Write at least six rules. Include at least one rule that handles unknown sensor information and one rule that handles conflicting evidence.

For example, a high-confidence obstacle reading on the planned route should trigger re-planning. If the route is blocked and no valid alternate path exists, the assistant should request help. If battery reserve is low, it should avoid starting a new delivery. If a sensor is unavailable, the system should not assume that the route is clear.

Implement forward chaining or a simple rule engine. Display the facts used, rules activated, conclusion reached, and any missing evidence. Create test cases for normal behavior, low battery, a blocked corridor, stale observations, unavailable communication, and contradictory map and sensor information. A good demonstration includes both successful recommendations and cases where the system appropriately says it cannot decide.

For each test, record expected behavior before running the program. This prevents you from changing the expected answer after seeing the output. Ask a classmate to inspect your rules and identify a case you forgot. Update the knowledge base and explain why the rule change improves coverage without creating a new conflict.

8. Common mistakes

A frequent mistake is to assume that a conclusion is true simply because a rule derived it. The conclusion follows from the encoded premises, but the premises may be outdated, incomplete or false. Inference is not a substitute for evidence quality.

Another mistake is to use a rule that is too broad. “If an obstacle is detected, stop” may be safe in some situations but can cause unnecessary immobilization if the obstacle is far away or outside the robot’s path. Conditions should reflect the variables that matter, and the system should be tested at boundary cases.

A third mistake is to encode assumptions as facts. “Every corridor is accessible” is not an observation. Mark assumptions explicitly and test their consequences.

A fourth mistake is to treat absent information as negative information. A missing record about a door does not prove that the door is closed or open. Decide whether the application requires an explicit unknown state.

A final mistake is to create many rules without checking interactions. Rules can overlap, contradict one another or produce cycles. Maintain traceable rule identifiers, test combinations, and document priorities and stopping conditions.

9. Knowledge check and assignment

Explain the difference between a datum, a fact, a rule and a model. What is the difference between forward chaining and backward chaining? Why can a missing fact not always be treated as false? How might a timestamp change a rule’s conclusion? What should a system do when two sources disagree about whether a corridor is clear?

For the assignment, submit the working rule-based assistant, a diagram of its knowledge representation, a table of rules, and at least eight test cases. For every conclusion, show which facts and rules support it. Include at least one case where the result is “unknown; request more evidence.” Discuss how the system could be integrated with a learned perception component while keeping safety-critical decisions inspectable.

Module summary

Knowledge representation gives an AI system a structured way to store facts and derive conclusions. We examined propositions, objects, relations, graphs, rules, forward chaining and backward chaining. We also saw that information can be missing, stale or inconsistent, and that unknown should not automatically mean false. A rule-based system can explain which premises led to a recommendation, but its reliability depends on the quality and scope of the knowledge it contains.